

MuLaConf: Multi-label Conformal Prediction

Kostas Katsios

Harris Papadopoulos

*Computational Intelligence Research Lab.,
Frederick University, Nicosia, Cyprus*

*Machine Learning Research Group,
Albourne Partners (Cyprus) Ltd, Nicosia, Cyprus*

K.KATSIOS@ALBOURNE.COM

H.PAPADOPOULOS@FREDERICK.AC.CY

Editor: Ernst Ahlberg, Ulf Johansson, Henrik Boström, Alberto Carlevaro, Johan Hallberg Szabadváry and Lars Carlsson

Abstract

This paper presents **MuLaConf**, a Scikit-Learn-compatible Python package that implements Inductive (or Split) Conformal Prediction for multi-label classification tasks. Conformal Prediction is a rigorous machine learning framework for constructing prediction regions with coverage guarantees under the minimal assumption of data exchangeability. In multi-label classification, the Powerset Scoring approach explicitly computes p-values for all possible label combinations. However, it often yields uninformatively large prediction sets. **MuLaConf** addresses this by defining nonconformity measures in the error vector space. This allows users to utilize either the Mahalanobis distance to capture label correlations or the standard Euclidean Norm. Furthermore, the package incorporates structural penalties, based on Hamming distance and label-set cardinality, to significantly reduce prediction set sizes while strictly maintaining theoretical coverage. To overcome the exponential computational complexity of the powerset space, **MuLaConf** leverages PyTorch-accelerated tensor operations and a memory management mechanism. Finally, its modular architecture features lazy evaluation, enabling the on-the-fly updating of distance measures and penalty weights without the need to retrain the underlying machine learning models. Together, these features make **MuLaConf** a practical tool for reliable uncertainty quantification in multi-label settings and a foundation for future research in Conformal Prediction.

Keywords: Conformal Prediction, Multi-label Classification, Mahalanobis NonConformity Measure, Norm NonConformity Measure, Structural Penalties, Prediction Regions, Scikit-Learn, PyTorch.

1. Introduction

Multi-Label Learning (MLL) is a machine learning setting where each instance is associated with a set of labels, called a label-set. In particular, for a label space $\mathcal{L} = \{\lambda_1, \dots, \lambda_c\}$, where c denotes the total number of labels, all possible label-sets form the powerset $\mathcal{P}(\mathcal{L})$.

Conformal Prediction (CP) (Vovk et al., 2005) is a machine learning framework for producing prediction regions with coverage guarantees without assuming anything more than data exchangeability. The framework is highly adaptable, and its Inductive (or Split) form can be combined with any conventional method while requiring the underlying model to be trained only once.

A number of CP approaches have been proposed for the MLL setting. Papadopoulos (2026) reviews these approaches and proposes a taxonomy based primarily on the level at

which nonconformity scores and p-values are computed, distinguishing three primary categories: label-set-level scoring, per-label scoring, and learned set-function scoring. It also proposes clarifying naming conventions for some potentially misleading names used in prior work. MuLaConf implements the label-set-level approach termed Powerset Scoring CP (PS-CP) in that review; for further details on the taxonomy, the other MLL-CP approaches, and the proposed naming conventions, see [Papadopoulos \(2026\)](#). PS-CP was introduced by [Papadopoulos \(2014\)](#), with a substantial computational-efficiency improvement for its L_p -norm version later proposed by [Maltoudoglou et al. \(2022\)](#). PS-CP directly computes nonconformity scores and p-values for candidate label-sets, providing valid label-set coverage. Direct evaluation over the entire powerset, however, can quickly become computationally intractable as the number of labels grows. Furthermore, PS-CP can produce uninformatively large prediction regions, limiting its practical utility. To improve region informativeness, [Katsios and Papadopoulos \(2024\)](#) introduced the Mahalanobis distance in the error vector space to account for label correlations, while [Katsios and Papadopoulos \(2025\)](#) incorporated structural penalties based on Hamming distance and label-set cardinality to produce tighter valid prediction regions.

To make the methods from these recent publications widely accessible, this paper introduces the MuLaConf Python package. MuLaConf implements Inductive CP (ICP) for multi-label tasks by bridging the widely known Scikit-Learn framework with the computational power of PyTorch. It integrates these structural penalties and defines nonconformity measures in the error vector space, allowing users to choose between the Mahalanobis distance or the standard Euclidean Norm. For ensuring positive definiteness, the covariance matrix is estimated using Ledoit-Wolf shrinkage. Designed with a modular, lazy evaluation architecture, MuLaConf permits users to update distance measures and penalty weights on-the-fly. These updates avoid retraining the underlying machine learning models, while recalculating only the conformal components required by the modified configuration. The package provides a unified wrapper interface (`ICPWrapper`) for integration with any Scikit-Learn compatible multi-label classifier, as well as lower-level interfaces (`InductiveConformalPredictor` and `PredictionRegions`) for retrieving and evaluating the resulting prediction sets.

There exist Python libraries implementing various CP techniques, such as `puncc` ([Mendil et al., 2023](#)), `MAPIE` ([Cordier et al., 2023](#)), `TorchCP` ([Huang et al., 2025](#)) and `crepes` ([Boström, 2022](#)). However, to the best of our knowledge, none of these libraries currently supports multi-label classification.

A defining feature of MuLaConf is its ability to handle the $\mathcal{O}(2^c)$ exponential complexity of the powerset space through PyTorch-accelerated tensor operations and memory management. The package utilizes GPU acceleration to handle the heavy mathematical workloads associated with evaluating all 2^c combinations. This acceleration specifically targets the most computationally demanding parts of the framework: the precalculation of the structural penalties and the core nonconformity scoring phase, where large tensors are processed to calculate distances for all label-set combinations simultaneously. To prevent Out-Of-Memory (OOM) errors during these intense matrix multiplications or the subsequent prediction region extraction phase, MuLaConf employs a batching procedure. By adjusting threshold parameters based on system RAM or VRAM, the package safely batches the

computations, ensuring that the system can smoothly process complex multi-label datasets while strictly maintaining the mathematical precision required by the conformal framework.

The remainder of this paper presents MuLaConf’s mathematical background, functionality and architecture. Section 2 defines the mathematical formulation of the implemented approach. Section 3 explains the package installation and dependencies, while Section 4 outlines the data partitioning steps. Section 5 details the Scikit-Learn wrapper interface, including model fitting, classification strategy switching, and on-the-fly parameter updates. Section 6 introduces the standalone `InductiveConformalPredictor` for users integrating custom classification architectures. Section 7 discusses the extraction of valid prediction regions and the evaluation of conformal metrics. Section 8 details the package’s internal memory management system, which ensures computational stability when dealing with exponential label spaces. Section 9 presents an experimental benchmarking of the framework’s execution speed, prediction region reduction, and empirical validity. Finally, Section 10 concludes the work.

2. Implemented Approach

The MuLaConf package implements the PS-CP approach within the ICP framework, which is based on the partitioning of the dataset into a *proper-training* set and a *calibration* set. An underlying classification model is trained on the proper-training instances to produce predicted probabilities. The core concept of the PS-CP approach is to calculate a non-conformity score for all candidate label-sets. The nonconformity scores derived from the calibration set are used to determine the p-value for a multi-hot label-set vector. Any candidate label-set whose p-value is greater than a specific significance level is included in the prediction region.

In MLL the set of all possible label-sets corresponds to $\mathcal{P}(\mathcal{L})$, containing all 2^c possible subsets of $\mathcal{L}$. We represent each possible label-set by a multi-hot vector $\mathbf{y} = (y_1, \dots, y_c)$, forming the vector space $Y = \{0, 1\}^c$, which is equivalent to the powerset.

Let $\mathcal{Z} = \{(\mathbf{x}_i, \mathbf{y}_i) : \mathbf{x}_i \in \mathbb{R}^s, \mathbf{y}_i \in Y\}$ denote a dataset, with n instances, s features and c classes, pairing the features $\mathbf{x}_i = (x_{i_1}, \dots, x_{i_s})$ and the label-sets $\mathbf{y}_i = (y_{i_1}, \dots, y_{i_c})$. The dataset is partitioned into a proper-training set $\mathcal{Z}_{tr} = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1, \dots, q}$, where q is the number of proper-training instances, and a calibration set $\mathcal{Z}_{cal} = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=q+1, \dots, n}$.

For any input instance $\mathbf{x}_i$, the underlying model $\mathbf{o} : \mathbb{R}^s \rightarrow \mathbb{R}^c$ produces a vector of probabilities $\mathbf{o}(\mathbf{x}_i) = (o_1, \dots, o_c)$, where $o_k \in [0, 1]$ denotes the probability that the instance belongs to class k , for $k = 1, \dots, c$.

The package computes the absolute residual between a candidate label-set $\mathbf{y}_i$ and the model output $\mathbf{o}(\mathbf{x}_i)$ to form the error vector $\mathbf{r}_i : Y \times \{\mathbf{o}(\mathbf{x}_i)\} \rightarrow \mathbb{R}^c$, written as,

$$\mathbf{r}_i = |\mathbf{y}_i - \mathbf{o}(\mathbf{x}_i)|. \quad (1)$$

These vectors constitute the c -dimensional error vector space and serve as the primary inputs for the distance-based nonconformity measure.

The nonconformity score for a pair $(\mathbf{x}_i, \mathbf{y}_i)$ is defined as $a_i = A(\mathbf{x}_i, \mathbf{y}_i)$, where $A : \mathcal{Z} \rightarrow \mathbb{R}$ is the nonconformity measure, calculated as the weighted sum of the normalized chosen distance measure and the structural penalties:

$$A(\mathbf{x}_i, \mathbf{y}_i) = \frac{d(\mathbf{r}_i)}{d_{max}} + \mu \cdot Hp(\mathbf{y}_i) + \nu \cdot Cp(\mathbf{y}_i), \quad (2)$$

where $d(\mathbf{r}_i)$ is the distance metric, d_{max} denotes the corresponding normalization factor, i.e., $d_{N_{max}}$ for the Euclidean Norm and $d_{M_{max}}$ for the Mahalanobis distance, $Hp(\mathbf{y}_i)$ is the Hamming penalty, $Cp(\mathbf{y}_i)$ is the Cardinality penalty, μ and ν are non-negative real values that serve as weights for the structural penalties.

Users can choose between two distance measures for the nonconformity score:

- **Euclidean Norm:** The Norm distance measure of an error vector $\mathbf{r}_i = (r_1, \dots, r_c)$ is defined as,

$$d_N(\mathbf{r}_i) = \sqrt{r_1^2 + \dots + r_c^2}. \quad (3)$$

The normalization factor for the Euclidean Norm is written as $d_{N_{max}} = \sqrt{c}$, and the term $d_N(\mathbf{r})/d_{N_{max}}$ lies in $[0, 1]$, since $\mathbf{r} \in [0, 1]^c$.

- **Mahalanobis Distance:** Utilizes the inverse covariance estimate Σ^{-1} from the proper-training set to account for label correlations. The Mahalanobis distance measure of an error vector $\mathbf{r}_i$ is defined as,

$$d_M(\mathbf{r}_i) = \sqrt{\mathbf{r}_i^T \Sigma^{-1} \mathbf{r}_i}. \quad (4)$$

The covariance matrix $\bar{\Sigma}$ is calculated using the error vectors $\mathbf{r}_i$, where $i = 1, \dots, q$, derived from the proper-training set. This matrix captures the label-wise correlation structure of the model's residuals, defined as,

$$\bar{\Sigma} = \frac{1}{q-1} \sum_{i=1}^q (\mathbf{r}_i - \bar{\mathbf{r}})(\mathbf{r}_i - \bar{\mathbf{r}})^T, \quad (5)$$

where $\bar{\mathbf{r}}$ denotes the mean error vector of the proper-training set. In **MuLaConf**, the covariance matrix $\bar{\Sigma}$ is regularized using Ledoit-Wolf shrinkage (Ledoit and Wolf, 2004), producing a well-conditioned covariance estimate Σ . When the proper-training error vectors exhibit nonzero empirical variance, the Ledoit-Wolf estimate is positive definite, i.e., $\Sigma \in S_{++}^c$,¹ and therefore invertible, as required for the Mahalanobis distance calculation. The normalization factor is defined as

$$d_{M_{max}} = \sqrt{\mathbf{1}^T |\Sigma^{-1}| \mathbf{1}}, \quad (6)$$

where $\mathbf{1} = (1, \dots, 1)$ and $|\Sigma^{-1}|$ denotes the matrix obtained by taking the absolute value of each entry of Σ^{-1} . Since every error vector satisfies $\mathbf{r} \in [0, 1]^c$, this choice provides a computationally efficient upper bound for the Mahalanobis distance. Specifically,

$$\mathbf{r}^T \Sigma^{-1} \mathbf{r} \leq \mathbf{r}^T |\Sigma^{-1}| \mathbf{r} \leq \mathbf{1}^T |\Sigma^{-1}| \mathbf{1}. \quad (7)$$

Therefore,

$$d_M(\mathbf{r}) \leq d_{M_{max}}, \quad (8)$$

and the normalized Mahalanobis term $d_M(\mathbf{r})/d_{M_{max}}$ lies in $[0, 1]$. This avoids the computationally expensive search over all possible error vectors for the exact maximum distance. The two penalties are defined as:

1. We use the notation S_{++}^c to denote the set of symmetric positive definite matrices,

$$S_{++}^c = \{\Sigma \in S^c \mid \Sigma \succ 0\},$$

where $S^c = \{\Sigma \in \mathbb{R}^{c \times c} \mid \Sigma = \Sigma^T\}$.

- The **Hamming penalty** (Hp), which is the minimum value of the normalized Hamming distances (using the XOR operator $\oplus$) between a label-set $\mathbf{y} \in Y$ and every proper-training label-set $\mathbf{y}_i \in \mathcal{Z}_{tr}$, where $i = 1, \dots, q$,

$$Hp(\mathbf{y}) = \min_{i=1, \dots, q} \left(\frac{1}{c} \sum_{k=1}^c y_{ik} \oplus y_k \right). \quad (9)$$

The Hamming penalty quantifies how much a candidate label vector differs from the label-sets observed in the proper-training data.

- The **Cardinality penalty** (Cp) of a label-set $\mathbf{y} \in Y$ is defined as,

$$Cp(\mathbf{y}) = 1 - f_{card}(Card(\mathbf{y})), \quad (10)$$

where the cardinality of a label vector $\mathbf{y} \in Y$ is given by,

$$Card(\mathbf{y}) = \sum_{k=1}^c y_k, \quad (11)$$

and $f_{card} : \{0, \dots, c\} \rightarrow [0, 1]$ denotes the empirical frequency distribution of cardinalities in the proper-training label vectors $\mathbf{y}_i \in \mathcal{Z}_{tr}$, defined as,

$$f_{card}(b) = \frac{1}{q} \sum_{i=1}^q \mathbb{I}[Card(\mathbf{y}_i) = b], \quad \text{for } b = 0, \dots, c, \quad (12)$$

where $\mathbb{I}[\cdot]$ denotes the indicator function, which returns 1 if the condition is true and 0 otherwise. The Cardinality penalty penalizes candidate label-sets whose number of active labels differs from the cardinalities observed in the proper-training set.

Note that each component (2), namely $\frac{d(\mathbf{r})}{d_{max}}$, $Hp(\mathbf{y})$, and $Cp(\mathbf{y})$, takes values in $[0, 1]$. Their empirical distributions, however, depend on the structure of the dataset.

Finally, MuLaConf computes all the p-values for all the possible label-sets $\mathbf{y} \in Y$ to construct the prediction regions. Denote a test instance as $\mathbf{x}_*$, with its nonconformity score denoted as $a_{n+1}(\mathbf{x}_*, \mathbf{y})$, then the p-value is defined as,

$$p(\mathbf{y}) = \frac{|j = q + 1, \dots, n : a_j(\mathbf{x}_j, \mathbf{y}_j) \geq a_{n+1}(\mathbf{x}_*, \mathbf{y})| + 1}{n - q + 1} \quad (13)$$

The prediction region for a given significance level $\varepsilon \in [0, 1]$ of the test instance $\mathbf{x}_*$ is formed as,

$$\tilde{\Gamma}_{\mathbf{x}_*}^\varepsilon = \{\mathbf{y} \in Y \mid p(\mathbf{y}) > \varepsilon\} \cup \left\{ \arg \max_{\mathbf{y} \in Y} (p(\mathbf{y})) \right\}. \quad (14)$$

Proposition 1 *Let $\Gamma_{\mathbf{x}_*}^\varepsilon = \{\mathbf{y} \in Y \mid p(\mathbf{y}) > \varepsilon\}$ be a valid conformal prediction region such that $P(\mathbf{y}_* \in \Gamma_{\mathbf{x}_*}^\varepsilon) \geq 1 - \varepsilon$, where $\mathbf{y}_*$ denotes the true label-set and P denotes the probability of coverage. For any candidate label-set $\mathbf{y} \in Y$, the extended region $\tilde{\Gamma}_{\mathbf{x}_*}^\varepsilon = \Gamma_{\mathbf{x}_*}^\varepsilon \cup \{\mathbf{y}\}$ remains valid, satisfying $P(\mathbf{y}_* \in \tilde{\Gamma}_{\mathbf{x}_*}^\varepsilon) \geq 1 - \varepsilon$.*

Proof Since $\Gamma_{\mathbf{x}_*}^\varepsilon \subseteq \tilde{\Gamma}_{\mathbf{x}_*}^\varepsilon$, it holds that $P(\mathbf{y}_* \in \Gamma_{\mathbf{x}_*}^\varepsilon) \leq P(\mathbf{y}_* \in \tilde{\Gamma}_{\mathbf{x}_*}^\varepsilon)$. By assumption, the original region $\Gamma_{\mathbf{x}_*}^\varepsilon$ satisfies the coverage guarantee $P(\mathbf{y}_* \in \Gamma_{\mathbf{x}_*}^\varepsilon) \geq 1 - \varepsilon$. Therefore,

$P(\mathbf{y}_* \in \tilde{\Gamma}_{\mathbf{x}_*}^\varepsilon) \geq 1 - \varepsilon$. The inclusion of the label-set with the maximum p-value, $\mathbf{y}_{max} = \arg \max_{\mathbf{y} \in Y} p(\mathbf{y})$, as shown in (14), is a specific case of this expansion, where $\mathbf{y} = \mathbf{y}_{max}$. ■

In the implementation, this non-empty prediction region correction is optional and is controlled through the `non_empty_prediction_regions` property of the returned `PredictionRegions` object. By default, this property is set to `True`, so the package includes the label-set with the maximum p-value whenever the conformal prediction region is empty. Users may disable this functionality by setting this property to `False` before extracting prediction regions.

3. Installing and Importing MuLaConf

The source code of MuLaConf is openly available on GitHub², with comprehensive documentation hosted on Read the Docs³. The package can be installed directly from the Python Package Index (PyPI) using the standard Python package manager. The command, `pip install mulaconf` installs the most recent version of the package. Alternatively, for those using the Conda ecosystem, the package can be installed with the command, `conda install -c conda-forge mulaconf`.

The package exposes three primary classes:

- `ICPWrapper`, which provides a high-level Scikit-Learn compatible interface for managing the model training and inference.
- `InductiveConformalPredictor`, which handles the lower-level conformal prediction nonconformity scoring.
- `PredictionRegions`, which serves as a container for extracting prediction sets at specific significance levels and evaluating their empirical performance.

Once installed, MuLaConf can be imported into any Python script or interactive environment. The package supports two primary workflows: either executing the pipeline through the wrapper, or using the standalone predictor by directly providing the predicted probabilities for the classification task.

In the first case, the `ICPWrapper` is designed to bridge Scikit-Learn and PyTorch. A typical workflow involves importing the desired multi-label base classifiers and the PyTorch library alongside the core wrapper component of MuLaConf:

```
from mulaconf.icp_wrapper import ICPWrapper
```

In the second case, the `InductiveConformalPredictor` class is used as a standalone component. This class forms the core of the multi-label conformal prediction framework, supported by the computational efficiency of the PyTorch tensor operations:

```
from mulaconf.icp_predictor import InductiveConformalPredictor
```

Note that both of these approaches return an instance of the `PredictionRegions` container to extract and evaluate the results.

With the package successfully installed and the required dependencies imported, the user can proceed to partition the data, define the base classification strategy, and initialize the conformal predictor. These steps are detailed in the subsequent sections.

2. [GitHub](#)

3. [ReadTheDocs](#)

4. Data Preparation and Partitioning

Following standard ICP methodology, the available training data must be partitioned into a proper-training set and a calibration set. The proper-training set is used to fit the underlying multi-label classifier and define the structural dependencies of the label space, while the calibration set is reserved strictly for computing the nonconformity scores of known examples.

Data preparation requires separating the input features from their corresponding multi-hot label vectors. To operate within the error vector space introduced by [Papadopoulos \(2014\)](#), the labels must be represented as multi-hot vectors. This format is essential for the calculation of error vectors.

The package supports a wide range of input formats, accepting an array-like structure, including PyTorch tensors, NumPy arrays, Pandas DataFrames or Series, and standard Python lists. Internally, the data are automatically converted to tensors and moved to the specified computation device (CPU or GPU) for efficient processing.

5. Wrapping Scikit-Learn Classifiers

The `ICPWrapper` class is a user-friendly component of the `MuLaConf` package, specifically designed to interface with the Scikit-Learn ecosystem. This wrapper abstracts the underlying conformal prediction pipeline, managing the workflow of both the multi-label base classifier and the inductive conformal predictor.

5.1. Wrapper Initialization

The wrapper is flexible and accepts any Scikit-Learn compatible base estimator that implements the standard `fit` and `predict_proba` methods. For multi-label classification tasks, users typically employ either a standalone estimator or one wrapped within a meta-learner, such as a `MultiOutputClassifier` or a `ClassifierChain`.

The `ICPWrapper` is initialized with the chosen classification strategy and, optionally, the preferred distance measure, structural penalty weights, and the computation device. Specifically, the `classification_strategy` parameter accepts the core multi-label estimator responsible for generating the initial probability estimates. The `measure` (str: 'mahalanobis' or 'norm') parameter defines the mathematical framework for the non-conformity scores, allowing users to select either the label-correlated Mahalanobis distance or the standard Euclidean Norm. Structural penalties can be applied by assigning positive scalars to `weight_hamming` and `weight_cardinality`, which respectively penalize unseen label-set combinations and infrequent label-set sizes based on the proper-training data. Finally, the `device` (str: 'cpu' or 'cuda') argument provides explicit control over hardware acceleration, allowing the heavy tensor computations to be routed to the CPU or GPU. The default values are `measure='mahalanobis'`, `weight_hamming=0.0`, `weight_cardinality=0.0`, `device='cpu'`. Upon initialization, the wrapper caches these configuration arguments, which are essential for the subsequent computational pipeline of the package. The following code snippet demonstrates the workflow for initializing the conformal wrapper with a specific distance measure and penalty weights:

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.multioutput import MultiOutputClassifier
```

```

from mulaconf.icp_wrapper import ICPWrapper

# Define the base multi-label classification strategy
base_model = MultiOutputClassifier(RandomForestClassifier(n_estimators
=10))

# Initialize the MuLaConf wrapper
wrapper = ICPWrapper(
classification_strategy=base_model,
measure='norm',
weight_hamming=2.0,
weight_cardinality=1.5,
device='cpu'
)

```

5.2. Fit and Calibration

The ICP framework requires two operations, generating predicted probabilities and calculating calibration nonconformity scores. The wrapper manages these steps in two distinct phases: the `fit` and `calibrate` phases.

During the `fit` phase, the wrapper receives the features and multi-hot encoded labels of the proper-training set. It primarily performs two tasks. First, it trains the underlying Scikit-Learn classification strategy. Second, it initializes the `InductiveConformalPredictor` engine. It analyzes the unique label-set combinations and the cardinality frequencies present in the proper-training label-sets, subsequently computing the Hamming and Cardinality penalties, respectively. In addition, it calculates the error vectors using the predicted probabilities generated by the underlying classifier for the proper-training features (for details see Section 6).

To execute this phase, the `fit` method requires the proper-training data and optionally accepts arguments for the classifier. The `train_features` parameter expects an array-like structure representing the input space, with dimensions corresponding to the number of samples and features. Correspondingly, the `train_labels` parameter (array-like structure) requires the ground truth targets to be formatted as a multi-hot matrix, where each column represents a distinct class. Additionally, users can utilize the `**kwargs` interface to pass any specific configuration arguments directly to the underlying Scikit-Learn classifier’s internal training routine.

In the `calibrate` phase, the wrapper utilizes the `InductiveConformalPredictor` engine to compute the nonconformity scores for the calibration set. The method requires two array-like arguments, `calib_features` and `calib_labels`. The `calib_features` are passed to the underlying base model to generate probability estimates for the calibration instances, while `calib_labels` provides the true targets as a multi-hot matrix (see Subsection 6.2).

The following code snippet demonstrates the sequential execution of the `fit` and `calibrate` methods:

```

wrapper.fit(X_train, y_train)
wrapper.calibrate(X_calib, y_calib)

```

Beyond basic calibration, the `ICPWrapper` incorporates a model fingerprinting mechanism to track changes in the underlying classification strategy. If the base classifier or its

hyperparameters are modified, the wrapper detects the change during the next calibration phase and automatically retrain the model on the cached proper-training data. If such a change is detected during prediction, a `RuntimeError` is raised, requiring the user to rerun the fitting and calibration steps to preserve validity.

In addition, **MuLaConf** supports on-the-fly updates of the distance measure and structural penalty weights. Changes to `measure`, `weight_hamming`, or `weight_cardinality` trigger lazy recalibration of the conformal scores without retraining the underlying classifier. When the distance measure is changed, the covariance-based distance matrix is recomputed, while penalty-weight updates reuse the cached structural quantities. This enables efficient experimentation with different distance and penalty configurations.

Classification strategy switching and on-the-fly updates of the distance measure and structural penalty weights are described in Subsections 5.4 and 5.5, respectively.

5.3. Prediction Regions

Once the underlying classifier is fitted on the proper-training set and the calibration non-conformity scores are calculated, the system is ready to evaluate unseen instances. In the **MuLaConf** package, passing the test features to the wrapper’s `predict` method computes the p-values for all possible label combinations and returns an instance of the `PredictionRegions` class. The method requires a single argument, `test_features`, which accepts an array-like structure containing the input features for the test instances.

The `PredictionRegions` object acts as a dedicated container for the conformal prediction results. It stores the computed p-values and the corresponding label-set combinations, and it exposes a `non_empty_prediction_regions` property, which is set to `True` by default.

Users can extract the valid prediction regions by calling this object directly with a chosen significance level, typically denoted as ε (e.g., $\varepsilon = 0.1$ for a 90% confidence level), or a list of multiple significance levels. The resulting output is a collection of the valid multi-hot label-sets (represented as a list of PyTorch tensors) for each test instance that satisfy the confidence threshold. An instance of this object can be retrieved as follows:

```
prediction_obj = wrapper.predict(X_test)
prediction_regions = prediction_obj(significance_level=0.1)
```

For a comprehensive guide on extracting prediction regions, accessing the underlying p-values, modifying the `non_empty_prediction_regions` property, and evaluating empirical performance metrics such as coverage and prediction region size, readers are encouraged to consult Section 7.

5.4. Hyperparameter Management and Strategy Switching

A core design principle of the **MuLaConf** package is its adherence to the Scikit-Learn API, which extends to how the hyperparameters of the underlying base classifiers are managed. When initializing the `ICPWrapper`, the user passes a fully configured Scikit-Learn base estimator and is entirely responsible for defining the hyperparameters during the base model’s initialization.

Users can configure the underlying model’s parameters in two distinct ways. The first approach is direct initialization, where the base classifier is set up with its specific parameters prior to being passed to the wrapper, as demonstrated in the following example:

```

from sklearn.ensemble import RandomForestClassifier
from sklearn.multioutput import MultiOutputClassifier
from mulaconf.icp_wrapper import ICPWrapper

base_model = MultiOutputClassifier(RandomForestClassifier(n_estimators
=10))
wrapper = ICPWrapper(classification_strategy=base_model)
wrapper.fit(X_train_prop, y_train_prop)

```

Alternatively, arguments can be passed to the `fit` method as a dictionary. When utilizing meta-estimators (such as `MultiOutputClassifier` or `ClassifierChain`), parameters must be passed using the `'estimator_'` prefix to correctly route them to the inner model (e.g., `args = {'estimator_n_estimators': 5}`).

```

from sklearn.ensemble import RandomForestClassifier
from sklearn.multioutput import MultiOutputClassifier
from mulaconf.icp_wrapper import ICPWrapper

base_model = MultiOutputClassifier(RandomForestClassifier())
wrapper = ICPWrapper(classification_strategy=base_model)
args = {'estimator_n_estimators': 5}
wrapper.fit(X_train, y_train, **args)

```

Conversely, when using a standalone multi-label classifier, omit the prefix and pass the arguments directly (e.g., `args = {'n_estimators': 10}`).

Moreover, users frequently need to tune hyperparameters or completely swap out the base classification algorithm to optimize performance. To support this without requiring the user to manually reconstruct the entire conformal prediction pipeline, the `ICPWrapper` incorporates an internal mechanism for strategy switching. This feature is supported by a model fingerprinting system. When the base model is initially fitted on the proper training set, the wrapper computes and caches a secure hash (fingerprint) of the model's architecture, hyperparameters, and associated arguments. After fitting, if the wrapper detects a change (via fingerprinting) during the next calibration phase, it will automatically retrain the new model on the cached proper training set. Note that during the first execution of the calibration method, the calibration data is cached by the ICP predictor module, and subsequently, the user can call `calibrate()` without passing arguments to run recalculations on the cached data.

```

from sklearn.neighbors import KNeighborsClassifier
from sklearn.multioutput import ClassifierChain

wrapper.strategy = ClassifierChain(KNeighborsClassifier(n_neighbors=5))
wrapper.calibrate(X_calib, y_calib)

```

If the wrapper detects a change during the prediction procedure, it will raise a `RuntimeError` to stop execution. This requires the user to manually rerun the necessary `fit` and `calibrate` procedures, ensuring the statistical validity of the newly generated prediction regions is safely maintained.

This architectural flexibility ensures that `MuLaConf` easily integrates into automated hyperparameter optimization pipelines (such as Scikit-Learn's `GridSearchCV`) while pre-

serving the mathematical guarantees of the Conformal Prediction framework. We encourage users to consult the Scikit-Learn documentation for guidance on choosing appropriate classification strategies and tuning their corresponding hyperparameters.

5.5. Parameter Updates and Lazy Recalibration

Determining the optimal distance framework and the ideal impact of structural penalties for a specific multi-label classification task often requires extensive empirical tuning. A significant computational bottleneck in such experimentation is the cost of repeatedly re-training the underlying Scikit-Learn classifier or executing heavy base-model inference for every new configuration of the conformal predictor.

To address this, MuLaConf implements an update mechanism based on lazy evaluation. Users can adjust the mathematical framework by modifying the wrapper’s exposed properties: `measure` (switching between Euclidean Norm and Mahalanobis distance), `weight_hamming`, and `weight_cardinality`.

Changing these properties does not trigger an immediate calculation. Instead, the wrapper simply marks the parameters as updated. The actual mathematical computations remain pending until the user performs a new calibration or prediction. The system handles these updates through two distinct execution paths:

Explicit Recalibration: If the user modifies the parameters and manually calls the `calibrate` method without any arguments, the module immediately applies the pending updates. Because the package caches the underlying probability estimates and ground truth labels from the initial `fit` and `calibrate` phases, it completely bypasses the Scikit-Learn model inference. It then recalculates the structural dependencies, reforms the covariance matrix (if the distance measure was changed), and recalculates the calibration scores.

```
wrapper.measure = 'norm'
wrapper.weight_hamming = 1.0
wrapper.weight_cardinality = 0.5
wrapper.calibrate()
```

Implicit Recalibration: Alternatively, the user can modify the parameters and proceed directly to the `predict` method without explicitly calling the `calibrate` method. During the prediction phase, the module automatically detects the pending parameter updates. Before generating the new prediction regions for the test instances, it implicitly triggers the recalibration process internally, ensuring the nonconformity thresholds align with the newly defined parameters.

```
wrapper.measure = 'mahalanobis'
wrapper.weight_hamming = 2.0
wrapper.weight_cardinality = 1.0
updated_prediction_regions = wrapper.predict(X_test)(significance_level=0.1)
```

This architectural design allows users to rapidly explore the trade-offs between empirical coverage and statistical efficiency (such as the N-Criterion and S-Criterion) across different structural configurations and distance metrics with minimal computational overhead.

6. Standalone Inductive Conformal Predictor

While the `ICPWrapper` provides a convenient interface for Scikit-Learn users, the `MuLaConf` package is designed to be modular. At its core lies the `InductiveConformalPredictor` class, which serves as the standalone mathematical engine for the multi-label conformal prediction framework. This standalone interface is designed for users who manage their own model training pipelines or utilize non Scikit-Learn frameworks (e.g, PyTorch or XGBoost).

6.1. Inductive Conformal Predictor Initialization

The initialization of the standalone engine is the phase where the package executes its structural analysis of the label space. The `InductiveConformalPredictor` requires the user to provide precomputed predicted probabilities and ground truth label-sets for the proper-training set, alongside the distance measure and the structural penalty weights. Upon initialization, the engine executes the following operations:

- **Label-set Powerset Production:** It generates and caches all possible label-set combinations within the powerset (2^c combinations for c classes).
- **Structural Penalty Calculation:** Using the proper-training label-sets, it identifies unique combinations to build the Hamming penalty (indexed to the label-set powerset) and calculates the frequency distribution of label-set sizes for the Cardinality penalty. Both penalty structures are then cached for subsequent use.
- **Covariance Matrix Formation:** It computes error vectors from the true labels and the predicted probabilities of the proper-training data. Consequently, the engine computes and caches a Ledoit-Wolf covariance estimate of the proper-training error vectors.

The `InductiveConformalPredictor` accepts `predicted_probabilities` and `true_labels` in array-like structures. Beyond the required probability estimates and ground truth labels, the constructor allows for the selection of the distance framework via the `measure` parameter and the configuration of structural penalty weights through the `weight_hamming` and `weight_cardinality` parameters. Furthermore, the `device` argument enables users to leverage hardware acceleration, ensuring that the powerset computations remain efficient even as the number of classes increases.

Assume proper-train probabilities have been obtained from a custom multi-label classifier. The following code snippet demonstrates the initialization of the class:

```
from mulaconf.icp_predictor import InductiveConformalPredictor

icp = InductiveConformalPredictor(
    predicted_probabilities=train_probs,
    true_labels=train_labels,
    measure='norm',
    weight_hamming=2.0,
    weight_cardinality=1.5,
    device='cpu' )
```

To ensure computational stability, especially given the exponential nature of the power-set label space, the predictor incorporates an internal memory management system for the Hamming and Cardinality penalty calculations. It processes label-set combinations in man-

ageable batches, controlled by the internal `_CPU_MAX_COMBINATIONS` and `_GPU_MAX_COMBINATIONS` constants, actively preventing memory overflows on both the CPU and GPU (see Section 8).

6.2. Calibrate Method

The calibration phase calculates the nonconformity scores needed to establish a valid prediction threshold. The `calibrate` method achieves this by evaluating the calibration predicted probabilities against their true label-sets. A nonconformity score is calculated in two straightforward steps. First, the function computes a normalized distance score in the error vector space using the selected `measure`. Second, it applies the structural penalties by adding the weighted Hamming and Cardinality values to this distance score.

Following initialization, the predictor must be calibrated. The user calls the `calibrate` method by passing the calibration `probabilities` and `labels`, which can be formatted as array-like structures. Upon execution, the method internally caches the provided calibration data and calculates the sorted nonconformity scores required for future predictions.

```
icp.calibrate(calib_probs, calib_labels)
```

A defining feature of the `InductiveConformalPredictor` is its parameter update mechanism. Users can modify the distance `measure` or adjust the structural scaling factors, `weight_hamming` and `weight_cardinality`, directly as attributes of the initialized instance. Following such modifications, calling the `calibrate()` method without passing new data arguments triggers an internal recalculation that leverages the previously cached data. The specific execution path of this mechanism adapts based on the applied update:

- **Distance Measure Updates:** If the `measure` attribute is changed, the engine computes a new covariance matrix using the proper-training data cached during initialization, followed by a full recalculation of the base distance scores.
- **Penalty Weight Updates:** Conversely, if only the structural penalty weights are adjusted, the engine operates only on the second component of the nonconformity measure (2). Because the base distance scores are cached during the initial calibration call, the method strictly recalculates the applied structural penalties without reprocessing the distance metrics.

```
icp.measure = 'norm'
icp.weight_hamming = 1.0
icp.weight_cardinality = 0.5
icp.calibrate()
```

Consequently, this design bypasses the computationally expensive $\mathcal{O}(2^c)$ powerset generation. By isolating the update paths, it enables rapid, iterative experimentation over different configurations with minimal computational overhead.

6.3. Predict Method

The prediction phase is the final stage, where the calibrated thresholds are used to evaluate unseen test data. For a given test probability vector, the `predict` method evaluates every possible label-set combination within the powerset space (2^c). The function computes a score for each of these combinations, which is then compared against the calibration scores to derive the corresponding p-values. The returned `PredictionRegions` object can ensure

that extracted prediction regions are non-empty by including the candidate label-set with the maximum p-value whenever no label-set satisfies the selected significance threshold. This optional behavior is controlled by the `non_empty_prediction_regions` property and is enabled by default.

The method accepts the predicted probabilities for the test set through the `probabilities` argument, in the same flexible array-like formats supported by the initialization and calibration routines.

The `predict` method of the standalone `InductiveConformalPredictor` outputs the same `PredictionRegions` object as the wrapper. This ensures that regardless of the chosen workflow, the user has access to the container for extracting valid prediction sets.

```
prediction_obj = icp.predict(test_probs)
```

Section 7 provides a comprehensive guide on extracting the valid prediction sets, accessing the underlying p-values, and executing empirical performance evaluations, such as calculating coverage and prediction region sizes.

7. Extracting Prediction Regions and Evaluation

After generating the `PredictionRegions` object, the subsequent step is to extract the valid prediction sets and evaluate the predictor’s performance. This section details how users can interact with this unified container to retrieve prediction regions at specific significance levels, access the underlying p-values for all possible label-set combinations, and utilize the evaluation metrics.

To extract the valid prediction sets, the user simply calls the object with a desired significance level (e.g., `significance_level=0.1`):

```
prediction_obj = icp.predict(test_probs)
prediction_regions = prediction_obj(significance_level=0.1)
```

Users may modify the `non_empty_prediction_regions` property of the `PredictionRegions` object. When this property is set to `True`, which is the default, the label-set with the maximum p-value is included whenever no candidate label-set satisfies the selected significance threshold. Users may disable this option by setting `False` before extracting prediction regions.

```
prediction_obj = icp.predict(test_probs)
prediction_obj.non_empty_prediction_regions = False
prediction_regions = prediction_obj(significance_level=0.1)
```

Additionally, users can access the p-values computed for every possible combination via the object’s `p_values` attribute:

```
sample_p_values = prediction_obj.p_values
```

The calculations are executed in manageable batches, regulated by the hardware-specific `_CPU_MAX_COMBINATIONS` and `_GPU_MAX_COMBINATIONS` constants, actively preventing OOM errors during this exhaustive evaluation.

To evaluate the predictor’s performance, the generated `PredictionRegions` object provides a dedicated `evaluate` method. This procedure computes standard conformal metrics to assess both the validity and efficiency of the applied configuration. The specific

metrics returned are controlled via boolean flags (`True` or `False`). By optionally providing the ground truth test label-sets (`true_labelsets` - array-like) and significance level(s) (`significance_level` - float or list of floats), users can calculate the p-values of the true test label-sets (`return_true_label_p_value`), the empirical coverage (`return_coverage`), the average set size of the prediction regions (`return_n_criterion`), the sum of p-values (`return_s_criterion`), the average sum of the p-values of the false label-sets (`return_observed_fuzziness`), and the average number of false label-sets included in the prediction region (`return_observed_excess`). Note that, metrics that depend on the extracted prediction regions, such as coverage, the N-Criterion, and observed excess, are computed with respect to the current value of the `non_empty_prediction_regions` property. Upon execution, the method returns a dictionary containing the requested metrics.

```
metrics = prediction_obj.evaluate(
    return_true_label_p_value=False,
    return_coverage=True,
    return_n_criterion=True,
    return_s_criterion=True,
    return_observed_fuzziness=True,
    return_observed_excess=True,
    true_labelsets=y_test,
    significance_level=0.1 )

print(metrics)
{ 'coverage': 0.9008264462809917,
  'n_criterion': 858.8636363636364,
  's_criterion': 412.99029541015625,
  'observed_fuzziness': 412.31787109375,
  'observed_excess': 857.9636363636364 }
```

Consistent with the calibration phase, the `predict` method features a parameter update mechanism. If a user modifies the distance `measure` or adjusts the structural penalty weights (`weight_hamming` and `weight_cardinality`) after calibration, the engine tracks these state changes. Subsequent calls to `predict(test_probs)` trigger an optimized recalculation path before the new p-values are generated. If the distance measure has changed, the predictor first recalculates the covariance-based distance matrix and updates the calibration scores using the cached calibration data. If only the penalty weights have changed, the predictor updates the calibration scores using the cached normalized distance scores and structural penalties. The test nonconformity scores are then recomputed for the supplied test probabilities under the modified configuration, allowing for rapid experimentation.

```
icp.measure = 'norm'
icp.weight_hamming = 1.0
icp.weight_cardinality = 0.5
new_prediction_obj = icp.predict(test_probs)
new_prediction_regions = new_prediction_obj(significance_level=0.1)
```

8. Memory Management and Computational Stability

The PS-CP approach implemented in the package is characterized by exponential computational complexity, scaling at $\mathcal{O}(2^c)$. To prevent OOM failures during tensor operations,

the architecture incorporates a memory management system. This system initiates a batching procedure based on hardware constraints and the specific computational phase being executed.

8.1. Memory Approximation Formulas

The package divides the conformal prediction pipeline into two distinct phases, each with its own memory consumption approximation formula and batching limits:

- **Matrix Computation Phase (Scoring & Penalties):** During the calculation of distance scores and structural penalties, the predictor class constructs massive 3D `float32` tensors. While a single `float32` value consumes 4 bytes, PyTorch must simultaneously hold multiple intermediate tensors in memory (combinations, subtraction results, and absolute errors) to execute the operations. Applying a heuristic $4\times$ overhead multiplier, the memory size for this phase is approximated as:

$$\text{Memory}_{\text{math}} \approx \text{max_combinations} \times c \times 16 \text{ bytes}$$

- **Prediction Region Extraction Phase:** For the formation of prediction regions, the class loads the precalculated p-values (4 bytes) and generates a boolean mask (1 byte) for filtering. Consequently, the required memory is estimated proportionally:

$$\text{Memory}_{\text{region formation}} \approx \text{batch_size} \times 5 \text{ bytes}$$

These approximation formulas are used to determine the default batching parameters described in the next section.

8.2. Batching Thresholds and Hardware Scaling

To keep memory usage within these limits, the predictor uses adjustable constants to control the batch sizes.

During the computationally heavy phase of scoring the 2^c label-sets, `_CPU_MAX_COMBINATIONS` and `_GPU_MAX_COMBINATIONS` limit the maximum number of label-set combinations processed in a single pass. The default GPU threshold is set to 5×10^6 combinations for standard 8GB VRAM. The CPU threshold is kept lower, defaulting to 2×10^6 , to preserve system RAM for the operating system (OS).

Conversely, the prediction phase is controlled by `_REGION_BATCH_SIZE`. Because this phase relies on 1D boolean masking rather than 3D tensor multiplications, it processes a larger default batch size of 50×10^6 combinations for 8GB RAM systems.

Overall, tuning these constants involves a direct trade-off between execution speed and memory stability. Increasing the default thresholds accelerates processing but elevates the risk of OOM failures, whereas decreasing them ensures memory safety at the cost of longer computation times. Due to the exponential complexity of the powerset space, for users with 16GB of system RAM, we recommend limiting the application of this package to datasets with approximately 20 labels or fewer. For more details, users can check the `constants.py` file of the package on GitHub for more information about the adjustable constants and how to control the batch sizes.

8.3. Benchmarking and Cache Optimization

To prevent VRAM fragmentation during repetitive PS-CP, the package actively clears the CUDA memory cache by default. This safety mechanism is controlled via the `_EMPTY_CUDA_CACHE` boolean flag.

While crucial for stability, calling `torch.cuda.empty_cache()` forces the GPU to synchronize and communicate with the operating system to release memory blocks. This OS-level communication introduces time delays. To benchmark pure execution time, users with enough VRAM can set this flag to `False`. Bypassing this synchronization step removes OS delays, revealing the GPU’s actual processing speed.

9. Investigating Prediction Regions and Computational Efficiency

To evaluate the computational efficiency, execution time, and empirical validity of the package, experiments were run on two benchmark multi-label datasets, Yeast and TMC2007_500. These datasets were selected to represent a diverse range of label space sizes. The structural characteristics of each dataset are summarized in Table 1.

Table 1: Characteristics of the benchmark multi-label datasets.

Dataset	Instances	Attributes	Labels	Avg. Cardinality
Yeast	2,417	117	14	4.237
TMC2007_500	28,596	522	22	2.219

Due to the computational complexity associated with PS-CP, the experimental design was adjusted. For both datasets, a 10-fold cross-validation procedure was employed. In each fold, the data were partitioned into three distinct sets, proper-training, calibration and test. Specifically, 10% of the complete dataset was reserved for testing. Of the remaining data, 30% was used for calibration, while the rest was used as the proper-training set.

The experimental procedure was executed using a machine equipped with an Intel(R) Xeon(R) Gold CPU @ 2.9 GHz, 503 GB of RAM and an Nvidia A40 GPU (40GB VRAM). For the experiments, the underlying classifier was XGBoost.⁴ During the experiments, the package’s internal memory management constants were configured for standard hardware specifications, specifically systems equipped with 8GB of system RAM and 8GB of VRAM. Accordingly, the batching thresholds for evaluating label-set combinations were maintained at their default limits (`_CPU_MAX_COMBINATIONS`, `_GPU_MAX_COMBINATIONS`, and `_REGION_BATCH_SIZE`). Crucially, to accurately benchmark the pure mathematical processing speed of the hardware, the `_EMPTY_CUDA_CACHE` flag was set to `False`. Disabling this cache synchronization mechanism prevents artificial OS-level communication delays.

To evaluate the performance of the proposed package and method, we focus on three primary metrics, execution time (in seconds), empirical validity, and prediction region size (N-Criterion). For all experiments, the `non_empty_prediction_regions` property was set to its default value (`True`). All reported values represent the overall averages derived from the respective cross-validation procedures.

A primary objective of the proposed architecture is to overcome the computational bottlenecks inherent to PS-CP. To isolate the execution time of the conformal prediction

4. The experimental script, including the XGBoost configuration, is available on GitHub in the [examples/scripts](#) directory.

engine from the execution time of the underlying classifier, the experiments were executed using the standalone `InductiveConformalPredictor` class with precomputed probabilities obtained from the XGBoost classifier. Accordingly, the reported execution times exclude both the training and inference times of the underlying classifier and measure only the conformal prediction pipeline. We report timings for both a full conformal prediction run (the default case, Table 2) and recalibrations after penalty weight updates (Table 3).

Table 2: Average execution times (in seconds) for the default case.

Dataset	Device	Init	Calibrate	Predict	Evaluate	Total
Yeast	CPU	0.003	0.000	0.101	0.077	0.182
	CUDA	0.009	0.004	0.008	0.076	0.096
TMC2007_500	CPU	0.029	0.002	163.210	455.968	619.208
	CUDA	0.022	0.004	38.656	355.510	394.191

As presented in Table 2, the baseline execution times illustrate the exponential scaling behavior ($\mathcal{O}(2^c)$) associated with PS-CP. For the Yeast dataset, which possesses a moderate label space ($c = 14$), the framework rapidly processes the entire pipeline in under a quarter of a second on a standard CPU. However, as the number of labels increases to $c = 22$ in the TMC2007_500 dataset, the computational overhead of fully evaluating label combinations drives the CPU execution time up significantly, averaging over 619 seconds. Furthermore, the results highlight the distinct advantage of hardware acceleration. Utilizing the CUDA device results in a significant reduction in the prediction phase, where the main tensor computations are performed. The evaluation times also differ across the CPU and CUDA runs, reflecting the downstream cost of processing the generated p-values and prediction regions. Aside from the evaluation phase, which executes on the CPU, the execution times for obtaining prediction regions (Init, Calibrate, Predict) are 79.9% and 76.3% lower on the CUDA device compared to the CPU baselines for the Yeast and TMC2007_500 datasets, respectively. As detailed in Table 2, the initialization and calibration phases are nearly instantaneous. For smaller label spaces like Yeast, utilizing CUDA introduces a slight overhead during initialization due to the transfer of tensors to the GPU. However, this overhead is negligible compared with the significant speedup achieved during the prediction phase. Table 3 illustrates the computational efficiency of the recalibration mechanism after penalty

Table 3: Average execution times (in seconds) for penalty updates.

Dataset	Penalty	Device	Recalibration	Predict	Evaluate	Total
Yeast	Hamming	CPU	0.006	0.098	0.081	0.185
		CUDA	0.003	0.007	0.077	0.086
	Cardinality	CPU	0.000	0.100	0.073	0.173
		CUDA	0.000	0.007	0.073	0.080
TMC2007_500	Hamming	CPU	15.593	135.115	353.693	504.400
		CUDA	6.028	39.429	354.545	400.002
	Cardinality	CPU	0.001	127.804	313.580	441.386
		CUDA	0.001	39.224	318.513	357.737

weight updates across both datasets. For the moderate Yeast dataset, recalibration is almost instantaneous across all configurations. Even for the larger label space of the TMC2007_500 dataset, Cardinality updates take only milliseconds, while Hamming updates require several seconds, still taking substantially less time than recomputing all preprocessing steps

from scratch. This update path allows the system to transition rapidly into the prediction phase. Furthermore, consistent with the memory management batching process, utilizing the CUDA architecture during these penalty updates substantially reduces the prediction time for both datasets by approximately 93% for Yeast and 70% for TMC2007_500.

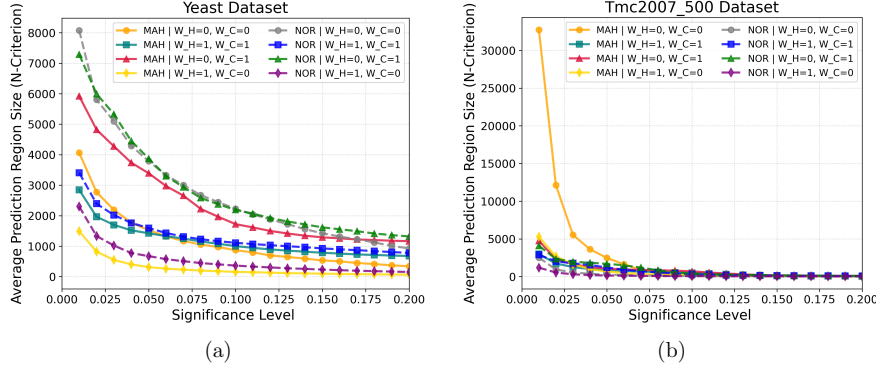


Figure 1: Average prediction set sizes across significance levels (ε) in the range $[0.01, 0.2]$ for the (a) Yeast and (b) TMC2007-500 datasets. Results are shown for both Mahalanobis and Norm measures, using weight combinations of 0 and 1 for the Hamming and Cardinality penalties.

Figure 1 presents the average prediction region size (N-Criterion) across significance levels in the range $[0.01, 2]$ for the benchmark datasets. The graphs illustrate the effectiveness of different structural penalty configurations (Hamming (W_H) and Cardinality (W_C) weights) combined with Mahalanobis (MAH) and Norm (NOR) distance measures.

Table 4: Percentage reduction against powerset.

level	Yeast		TMC2007_500	
	Max (%)	Min (%)	Max (%)	Min (%)
0.01	90.90	50.71	99.97	99.22
0.05	98.09	76.36	99.99	99.94
0.10	99.05	86.43	99.99	99.98
0.20	99.61	91.95	99.99	99.99

The label space of the Yeast dataset ($C = 14$) consists of 16,384 possible label-set combinations, while the TMC2007_500 dataset ($C = 22$) expands to over 4.1 million combinations. Table 4 details the percentage reduction of the prediction region sizes relative to these baseline powersets across specific significance levels.

Figure 2 presents the empirical coverage across the significance levels for both the Yeast and TMC2007_500 datasets. For small and moderate significance levels, all configurations align with the ideal calibration line, confirming the validity of the proposed framework. For larger significance levels, an overcoverage effect is observed, caused by the default `non_empty_prediction_regions` setting. Consequently, the coverage remains above the ideal diagonal at high significance levels.

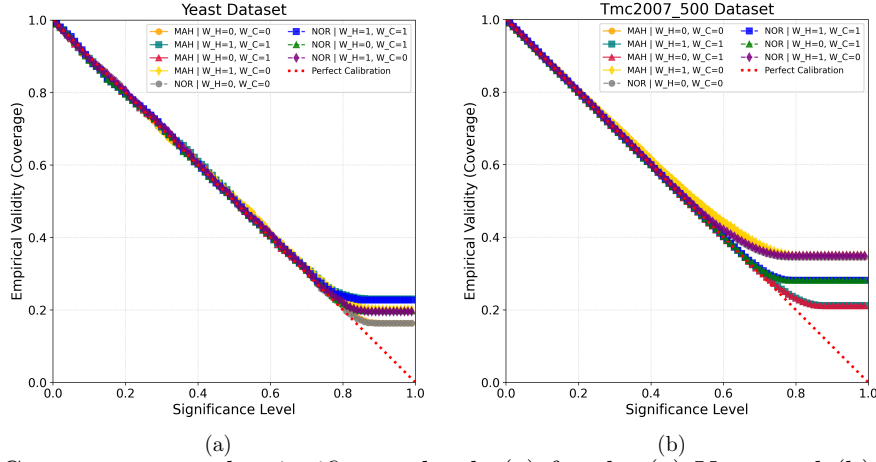


Figure 2: Coverage across the significance levels (ε) for the (a) Yeast and (b) TMC2007-500 datasets. Results are shown for both Mahalanobis and Norm measures, using weight combinations of 0 and 1 for the Hamming and Cardinality penalties.

10. Conclusion

This paper introduces **MuLaConf**, to the best of our knowledge, the first Python package implementing MLL CP. The package currently implements PS-CP, which assigns nonconformity scores and p-values at the label-set level and can account for label dependencies through the Mahalanobis distance and structural penalties. This is provided through a flexible, object-oriented architecture that supports nonconformity measures based on the Mahalanobis and Norm distance metrics combined with Hamming and Cardinality penalties.

A defining technical feature of **MuLaConf** is its integration with Scikit-Learn classifiers, coupled with a focus on computational efficiency. Recognizing the inherent bottlenecks of the PS-CP approach, the package supports GPU acceleration, significantly speeding up the prediction phase. Furthermore, the implementation of an internal caching mechanism allows for rapid recalibration following distance measure and penalty weight updates, without the need to retrain the classifier model. This optimized update path avoids unnecessary recomputation whenever possible.

However, it is important to acknowledge the fundamental hardware limitations imposed by the $\mathcal{O}(2^c)$ complexity of the PS-CP approach. Despite robust memory batching and caching optimizations, generating and evaluating the full powerset label space for datasets with a large number of labels may exceed the physical memory capacity of a system, causing it to run out of memory. Regardless of these physical hardware boundaries, **MuLaConf** achieves functional adaptability without compromising the theoretical guarantees inherent to Conformal Prediction.

Future work includes extending the package by implementing additional MLL CP approaches, such as Labelwise Score Aggregation CP (LSA-CP), referred to as Binary Relevance Multi-label CP in [Lambrou and Papadopoulos \(2016\)](#). Additionally, the integration of efficiency improvement approaches in the existing PS-CP implementation, such as the powerset label space reduction technique proposed by [Maltoudoglou et al. \(2022\)](#), will be investigated. This will further extend the practical applicability of **MuLaConf**.

References

- Henrik Boström. Crepes: A python package for generating conformal regressors and predictive systems. In *Conformal and Probabilistic Prediction with Applications*, pages 24–41. PMLR, 2022.
- Thibault Cordier, Vincent Blot, Louis Lacombe, Thomas Morzadec, Arnaud Capitaine, and Nicolas Brunel. Flexible and systematic uncertainty estimation with conformal prediction via the mapie library. In *Conformal and probabilistic prediction with applications*, pages 549–581. PMLR, 2023.
- Jianguo Huang, Jianqing Song, Xuanning Zhou, Bingyi Jing, and Hongxin Wei. Torchcp: A python library for conformal prediction. *Journal of Machine Learning Research*, 26(266):1–25, 2025.
- Kostas Katsios and Harris Papadopoulos. Multi-label conformal prediction with a mahalanobis distance nonconformity measure. *Proceedings of Machine Learning Research*, 230:522–535, 2024.
- Kostas Katsios and Harris Papadopoulos. Incorporating structural penalties in multi-label conformal prediction. *Proceedings of Machine Learning Research*, 266:230–249, 2025.
- Antonis Lambrou and Harris Papadopoulos. Binary relevance multi-label conformal predictor. In *Conformal and Probabilistic Prediction with Applications*, pages 90–104. Springer, 2016.
- Olivier Ledoit and Michael Wolf. A well-conditioned estimator for large-dimensional covariance matrices. *Journal of multivariate analysis*, 88(2):365–411, 2004.
- Lysimachos Maltoudoglou, Andreas Paisios, Ladislav Lenc, Jiří Martínek, Pavel Král, and Harris Papadopoulos. Well-calibrated confidence measures for multi-label text classification with a large number of labels. *Pattern Recognition*, 122:108271, 2022.
- Mouhcine Mendil, Luca Mossina, and David Vigouroux. Puncc: a python library for predictive uncertainty calibration and conformalization. In *Conformal and Probabilistic Prediction with Applications*, pages 582–601. PMLR, 2023.
- Harris Papadopoulos. A cross-conformal predictor for multi-label classification. In *Artificial Intelligence Applications and Innovations: AIAI 2014 Workshops: CoPA, MHDW, IIVC, and MT4BD, Rhodes, Greece, September 19-21, 2014. Proceedings 10*, pages 241–250. Springer, 2014.
- Harris Papadopoulos. Conformal prediction for multi-label learning: A review of methods and guarantees. *Philosophical Transactions of the Royal Society A*, 384:20250071, 2026. doi: 10.1098/rsta.2025.0071. In press.
- Vladimir Vovk, Alexander Gammerman, and Glenn Shafer. *Algorithmic learning in a random world*, volume 29. Springer, 2005.